■ AI Project Mentor — Fix Summary Report

Date & Time: 2026-11-03 17:36:21
Session Goal / Focus: Strict UB-Mode refactoring: remove duplication, simplify state, and improve readability
Total Fixes Applied: 7

---

1. Reuse patch and syntax validation helpers

Target File:	src/main.py
Category:	improvement
Why / ROI:	Eliminates ~35 lines of duplicated logic, prevents syntax checking drift, and guarantees uniform patch application across the bot.

Before (Original Code):

```
content = target_path.read_text(encoding="utf-8")
s_block = patch["search_block"]
r_fix = patch["ready_code_fix"]

normalized_content = content.replace("\r\n", "\n")
norm_s_block = s_block.replace("\r\n", "\n")
norm_r_fix = r_fix.replace("\r\n", "\n")

match_block = norm_s_block if norm_s_block and norm_s_block in normalized_content else (
norm_s_block.strip() if norm_s_block and norm_s_block.strip() in normalized_content else ""
)
if not match_block:
continue

match_fix = norm_r_fix if match_block == norm_s_block else norm_r_fix.strip()
new_content = normalized_content.replace(match_block, match_fix, 1)
target_path.write_text(new_content, encoding="utf-8")
applied_files.append(target_path)

syntax_error = None
if target_path.suffix == ".py":
try:
py_compile.compile(target_path, doraise=True)
except py_compile.PyCompileError as e:
syntax_error = str(e)
elif target_path.suffix == ".json":
try:
json.loads(target_path.read_text(encoding="utf-8"))
except Exception as e:
syntax_error = str(e)

if syntax_error:
target_path.write_text(content, encoding="utf-8")
await callback.message.answer
```

After (Applied Fix):

```
content = target_path.read_text(encoding="utf-8")
new_content = apply_text_patch(content, patch["search_block"], patch["ready_code_fix"])
if new_content is None:
continue

target_path.write_text(new_content, encoding="utf-8")
applied_files.append(target_path)

syntax_error = check_file_syntax(target_path)
if syntax_error:
target_path.write_text(content, encoding="utf-8")
await callback.message.answer(
f"■ <b>Syntax Error in {html.escape(patch['file_path'])}! Rollback applied.</b>\n"
f"<pre><code>{html.escape(syntax_error)}</code></pre>",
parse_mode="HTML"
)
return
```

2. Unify project state queries with helper

Target File:	src/database.py
Category:	improvement
Why / ROI:	Consolidates repetitive SQL queries into a single validated accessor, adhering to the Laziness Protocol.

Before (Original Code):

```
async def get_chat_id(self) -> Optional[int]:
"""Retrieves the registered Telegram chat_id."""
async with self._get_db() as db:
cursor = await db.execute("SELECT chat_id FROM project_state WHERE id = 1")
row = await cursor.fetchone()
return row[0] if row and row[0] else None

async def set_ping_interval(self, interval_hours: int):
"""Sets or updates the ping interval in hours."""
await self._update_state_field("ping_interval_hours", interval_hours)

async def set_user_verified(self):
"""Marks user as verified for filesystem access."""
await self._update_state_field("is_verified", 1)

async def get_ping_interval(self, default_hours: int = 24) -> int:
"""Retrieves the current reminder interval in hours."""
async with self._get_db() as db:
cursor = await db.execute("SELECT ping_interval_hours FROM project_state WHERE id = 1")
row = await cursor.fetchone()
return row[0] if row and row[0] else default_hours
```

After (Applied Fix):

```
async def _get_state_field(self, column: str, default: any = None) -> any:
"""Helper to fetch a single column from project_state for row id=1."""
allowed_columns = {
"chat_id", "ping_interval_hours", "is_verified", "summary",
"current_focus", "current_session_id", "ignored_count"
}
if column not in allowed_columns:
raise ValueError(f"Invalid state column: {column}")

async with self._get_db() as db:
cursor = await db.execute(f"SELECT {column} FROM project_state WHERE id = 1")
row = await cursor.fetchone()
return row[0] if row and row[0] is not None else default

async def get_chat_id(self) -> Optional[int]:
"""Retrieves the registered Telegram chat_id."""
return await self._get_state_field("chat_id")

async def set_ping_interval(self, interval_hours: int):
"""Sets or updates the ping interval in hours."""
await self._update_state_field("ping_interval_hours", interval_hours)

async def set_user_verified(self):
"""Marks user as verified for filesystem access."""
await self._update_state_field("is_verified", 1)

async def get_ping_interval(self, default_hours: int = 24) -> int:
"""Retrieves the current reminder interval in hours."""
return await self._get_state_field("ping_interval_hours", default_hours)
```

3. Simplify focus and verification queries

Target File:	src/database.py
Category:	improvement
Why / ROI:	Removes 12 lines of boilerplate database handling and unifies return fallback behavior.

Before (Original Code):

```
async def get_current_focus(self) -> str:
"""Retrieves the active session focus direction."""
async with self._get_db() as db:
cursor = await db.execute("SELECT current_focus FROM project_state WHERE id = 1")
row = await cursor.fetchone()
return row[0] if row and row[0] else "General Code Refactoring"
```

After (Applied Fix):

```
async def get_current_focus(self) -> str:
"""Retrieves the active session focus direction."""
return (await self._get_state_field("current_focus")) or "General Code Refactoring"
```

4. Deduplicate reverify keyboard with existing helper

Target File:	src/main.py
Category:	improvement
Why / ROI:	Deletes duplicate inline keyboard definitions and ensures all task completion buttons remain in sync.

Before (Original Code):

```
else:
await db.reset_ignored_count()
kb_reverify = InlineKeyboardMarkup(inline_keyboard=[
[InlineKeyboardButton(text="■ Verify Fix Again", callback_data=f"task:verify:{task_id}")],
[InlineKeyboardButton(text="■ Next Task", callback_data="quick:next")],
[InlineKeyboardButton(text="■ Stop Session & Get PDF Report", callback_data="task:stop_session")]
])
await status_msg.edit_text(
f"■■ <b>Verification Alert:</b>\n\n{feedback_escaped}\n\n"
f"<i>Update the file and click below to re-verify when ready:</i>",
parse_mode="HTML",
reply_markup=kb_reverify
)
```

After (Applied Fix):

```
else:
await db.reset_ignored_count()
await status_msg.edit_text(
f"■■ <b>Verification Alert:</b>\n\n{feedback_escaped}\n\n"
f"<i>Update the file and click below to re-verify when ready:</i>",
parse_mode="HTML",
reply_markup=get_post_task_keyboard(task_id)
)
```

5. Centralize focus menu presentation helper

Target File:	src/main.py
Category:	improvement
Why / ROI:	Removes duplicated Telegram messaging strings and centralizes focus menu rendering into a single handler.

Before (Original Code):

```
async def send_settings_menu(target: Message | CallbackQuery):
"""Sends the schedule settings menu for both command and quick action button callbacks."""
chat_id, send = resolve_target_channel(target)
await db.set_chat_id(chat_id)
hours = await db.get_ping_interval(config.PING_INTERVAL_HOURS)
text = (
f"■■ <b>Reminder Schedule Settings</b>\n\n"
f"Current interval: <b>every {hours} hours</b>.\n"
f"Select a new interval below:"
)
await send(text, parse_mode="HTML", reply_markup=get_settings_keyboard(hours))
```

After (Applied Fix):

```
async def send_settings_menu(target: Message | CallbackQuery):
"""Sends the schedule settings menu for both command and quick action button callbacks."""
chat_id, send = resolve_target_channel(target)
await db.set_chat_id(chat_id)
hours = await db.get_ping_interval(config.PING_INTERVAL_HOURS)
text = (
f"■■ <b>Reminder Schedule Settings</b>\n\n"
f"Current interval: <b>every {hours} hours</b>.\n"
f"Select a new interval below:"
)
await send(text, parse_mode="HTML", reply_markup=get_settings_keyboard(hours))

async def send_focus_menu(target: Message | CallbackQuery):
"""Sends the focused audit menu for both command and quick action button callbacks."""
chat_id, send = resolve_target_channel(target)
await db.set_chat_id(chat_id)
text = (
"■ <b>Focused Codebase Audit</b>\n\n"
"Choose a focus preset below, or simply <b>type any custom goal in the chat</b> "
"(e.g., <i>'Add rate limiting and protect database calls'</i>):"
)
await send(text, parse_mode="HTML", reply_markup=get_focus_presets_keyboard())
```

6. Standardize scanner logging output

Target File:	src/scanner.py
Category:	improvement
Why / ROI:	Guarantees file traversal warnings contain timestamps and proper log levels, avoiding unformatted stdout prints.

Before (Original Code):

```
except Exception as e:
print(f"Error reading file {rel_path}: {e}")
```

After (Applied Fix):

```
except Exception as e:
logging.warning(f"Error reading file {rel_path}: {e}")
```

7. Deduplicate report code block formatting

Target File:	src/report_generator.py
Category:	improvement
Why / ROI:	Eliminates duplicate string transformation logic and ensures consistent truncation limits across report sections.

Before (Original Code):

```
raw_search = (t.get('search_block') or 'N/A').strip()[:1500]
search_code_esc = html.escape(raw_search).replace('\n', '<br/>')

raw_fix = (t.get('ready_code_fix') or 'N/A').strip()[:1500]
fix_code_esc = html.escape(raw_fix).replace('\n', '<br/>')
```

After (Applied Fix):

```
def _format_code_snippet(code: str | None) -> str:
return html.escape((code or 'N/A').strip()[:1500]).replace('\n', '<br/>')

search_code_esc = _format_code_snippet(t.get('search_block'))
fix_code_esc = _format_code_snippet(t.get('ready_code_fix'))
```